

申高阳-二面作业-2026-04

图形前端工程师 二面 Vibe Coding 作业 — tldraw + R3F 3D in Canvas

作业背景

Meshy 是市占率第一的 AI 3D 生成平台，月访问量 500 万。我们正在探索的下一代创作形态是「3D in 2D Canvas」——用户在一个 tldraw 风格的无限画布上做创意编排，但每个“贴纸”内部其实是一个可交互的 3D 模型。这种形态结合了 2D 排版的自由度与 3D 内容的真实感，是介于 Figma / Miro 与传统 3D 编辑器之间的一种新工具范式。

参考产品：[Krea Realtime](#) ([Twitter 介绍](#))

⚠ 范围限定：只参考 Krea 实时画布里“图片转 3D”那一个小功能，不要看整体产品形态——范围铺太大反而失焦。

关注的交互特征只有两点：

1. 一张图片在画布上能被转换成一个 3D 模型对象
2. 转出来的 3D 模型在画布上像图片一样可以被拖动位置，但同时支持在原地左右旋转（绕 Y 轴）查看不同角度

我们这道题就是这个小功能在 tldraw + R3F 上的一种实现。

要把这件事做好，关键技术问题是：如何让 tldraw（2D 矢量画布）和 R3F（3D 场景）这两个独立的 React 渲染体系优雅地融合在一起？这是一个目前业内没有标准答案的方向。

这道题考察的是：

1. 未知方向的探索能力——能不能摸进 tldraw 自定义 shape 这条路，独立做出一个可工作的原型
2. 架构判断力——当 R3F Canvas 嵌入 tldraw shape 时，怎么权衡多个 WebGL Context vs 共享 Context、状态管理、坐标系统等问题
3. AI 协作能力——React / tldraw 如果你不熟，怎么用 AI Agent 把陌生栈的 ramp-up 成本压到最小

关于 Meshy API 的免费会员

本题需要调用 Meshy 的 Image-to-3D API，**API 调用消耗积分，免费账号默认积分有限**。我们为本次作业的候选人提供免费会员充值，流程如下：

1. 在 <https://www.meshy.ai/> 注册一个免费账号
2. 把**注册邮箱**回复给招聘 HR
3. HR 会把邮箱转给面试官，**我们会帮你充值免费会员**，足够本次作业使用
4. 充值到账后再开始动手，避免 API 额度卡壳影响节奏

充值通常在 1 个工作日内完成。如果你想先动手熟悉 tldraw / R3F 部分，可以先用本地预设 GLB 跑通主流程（见 Part 2），等会员到账后再接入 API。

作业要求

限时：4-6 小时

- 核心闭环建议在 2-3 小时内打通
- 剩余时间用于 polish、Vercel 部署、报告整理
- 请记录实际用时和每个阶段的大致时间分配

技术栈要求：

- React + TypeScript（必须）
- tldraw v3（必须，作为画布底座）
- React Three Fiber (R3F)（必须，作为 3D 渲染层）
- 自由选择构建工具（Vite 推荐）

工具使用：

- 全流程鼓励使用 AI Agent 加速（Claude Code / Cursor / Copilot / 任何你习惯的 AI 工具）
- 不只是写代码——文档查阅、源码阅读、架构选型、报告整理，任何你觉得 AI 能帮上忙的环节都可以用
- 请在 README 中记录：**每个环节用了什么 AI 工具、怎么用的、效果如何**。我们看重你怎么把 AI 作为探索陌生栈的杠杆

产品功能定义

为了减少歧义，先把目标产品形态写清楚：

1. **无限画布**：基于 tldraw v3，一张可平移、缩放的无限画布

2. **图片上传**：用户可以把本地图片拖入画布，或点击工具栏按钮上传——画布上呈现一个标准的 2D 图片 shape
 3. **图片转 3D**：图片 shape 上呈现一个小按钮（建议位于角落或选中后浮现），点击后调用 Meshy Image-to-3D API，原地把这张 2D 图片"升级"为一个 3D 模型 shape（保留位置 / 尺寸）
 4. **3D 模型在画布里被当作 2D 贴纸**：3D shape 的画布缩放、平移、删除走 tldraw 标准交互；3D 模型本身的姿态（绕 Y 轴旋转）通过 shape 上的独立旋转按钮调整
 5. **Undo / Redo**：上述所有操作（上传 / 转换 / 移动 / 缩放 / 姿态旋转 / 删除）都能被 tldraw 的 undo-redo 正确捕获
-

优先级清单（务必按此排序投入时间）

做不完非常正常。先确保 Must-have 跑通，再依次做 Nice-to-have。

● Must-have（核心信号，请优先保证）

- ☐ tldraw v3 跑起来，画布可平移、缩放
- ☐ 自定义 MeshyModelShape，shape 内部用 R3F 渲染加载到的 GLB 模型
- ☐ 图片 shape 上有一个小按钮，点击后用本地预设 **GLB 模拟「图片转 3D」**（替换为 MeshyModelShape）
- ☐ 3D shape 在画布上能像图片一样：被选中、拖动、等比缩放
- ☐ 3D shape 上有独立的旋转按钮，点击后模型绕 Y 轴旋转一定角度
- ☐ **Vercel 部署上线**，公开可访问

● Nice-to-have（加分项，做了显著加分但非必须）

- ☐ 真实接通 Meshy Image-to-3D API（loading / 进度 / 错误反馈）
- ☐ tldraw 自带 undo/redo 能完整回放：上传 → 转 3D → 移动 → 缩放 → 姿态旋转 → 删除
- ☐ 异步任务一致性：API 进行中用户移动 / 删除原图片 shape 时，逻辑不崩
- ☐ 画布缩放时 3D 渲染清晰度合理（DPR / 重渲染策略）
- ☐ 多 shape 场景下的性能方案（README 写思路即可，5+ shape 实测最好）

● Stretch（不期待做完，思路写到报告里同样得分）

- ☐ 共享 WebGL Context 方案
- ☐ 50+ shape 的扩展性设计
- ☐ 协同 / 持久化思考

我们更看重「Must-have 全部跑通 + 一份诚实的 README 说清你的取舍」，而不是“什么都做了一半”。

任务拆解

Part 1: 技术调研 (建议 25-40 min)

动手前先做三件事：

1. **Krea 实时画布“图片转 3D”扫读 (3-5 min)**：在 [Krea Realtime](#) 找到“图片转 3D”这个小交互，只关注两点：(a) 转换前后画布上对象的视觉过渡 (b) 3D 对象在画布上的拖动 + 原地旋转交互。把你的观察记一两句即可，**不要扩展到整体产品形态**
2. **tldraw 自定义 shape 机制调研**：tldraw v3 怎么扩展自定义 shape？ShapeUtil 的关键 API 是什么？shape 的 props（包括你自定义的字段）如何参与 undo/redo？shape 内部如何渲染自定义 React 组件、放置自定义按钮？
3. **R3F 嵌入 tldraw 的方案对比**：列出至少 2 种把 R3F 渲染嵌入 tldraw shape 的方案，对比 WebGL Context 数量、性能、状态同步、交互冲突等维度

AI 加速提示：tldraw 文档 (<https://tldraw.dev/>) 和 R3F 文档可以让 AI 帮你提炼关键 API。但方案选择必须是你自己的判断——把判断依据写进调研笔记，面试会追问。

产出：一份 200-400 字的调研笔记，写在 README 或单独 Markdown 中。明确说明你最终采用的方案和理由。

Part 2: 图片上传 + Image-to-3D API 接入 (建议 40-70 min)

2.1 图片上传 (Must-have)

- 用户能把本地 PNG / JPG 图片放到画布上：建议直接用 tldraw 自带的 image shape，或自己实现一个简易 image shape
- 上传方式至少支持：拖拽文件到画布 或 点击工具栏按钮选文件二选一即可

2.2 Meshy Image-to-3D API 接入 (Nice-to-have)

- API 文档：<https://docs.meshy.ai/api/image-to-3d>
- 调用 Image-to-3D 的标准流程：上传图片 → 创建任务（拿 task_id）→ 轮询任务状态 → 拿到 GLB URL
- API Key 通过环境变量注入（推荐 Vercel 环境变量），不要硬编码进仓库
- 鉴权和密钥管理可以走最简方案（本地代理或 dev-only env），不需要做生产级安全
- ⚠ Image-to-3D 一次轮询通常要 2-5 分钟，调试链路非常吃时间——先用 2.3 的本地兜底跑通主路径，再回头接真 API

2.3 兜底：本地预设 GLB (Must-have 路径)

- 在 `public/assets/` 放 2-3 个本地 GLB 文件，让"图片转 3D"按钮直接返回预设模型
- 兜底机制不影响评分——这是 Must-have 路径的推荐选择，先跑通主路径

AI 加速提示： API 鉴权、轮询逻辑、错误重试这类样板代码全交给 AI 写。这是题目里相对次要的一环，重点在 **Part 3**。

Part 3：核心实现 — 图片转 3D + tldraw 3D Shape (建议 100-140 min)

★ 重点

3.1 图片 shape 上的"转 3D"按钮 (~25 min, Must-have)

- 选中 (或 hover) 图片 shape 时，shape 上呈现一个小按钮 (角落或浮层均可)
- 点击后：
 - 进入 loading 状态 (视觉上图片 shape 显示进度 / 遮罩 / 骨架等任意合理反馈)
 - 调用 Image-to-3D API (或返回本地 GLB 兜底) 拿到 GLB URL
 - 在原图片 shape 同位置同尺寸 替换为自定义 `MeshyModelShape`，shape props 至少包含：`assetUrl`、`width`、`height`、`yRotation` (模型自身绕 Y 轴的旋转角度)

隐性考点 (Nice-to-have)： 你怎么处理"long-running 任务 + tldraw store 一致性"？任务进行中用户可能会移动 / 删除原图片 shape——你怎么处理？任务完成后能不能 undo 回到原图片状态？

3.2 R3F 渲染 (~40 min, Must-have) ★ 核心考察点

每个 `MeshyModelShape` 内部渲染加载到的 GLB 模型，关键点：

- 模型按 shape 的 `width × height` 自适应 fit
- 提供基础环境光 (环境贴图或简单 PBR 三点光) 让模型不发黑
- 透明背景，shape 边界外不绘制
- 画布缩放 (tldraw zoom) 时，shape 内的 3D 渲染清晰度需保持合理——这里隐藏一个 DPR / 重渲染策略的判断 (Nice-to-have)

隐性考点 (Nice-to-have)： 当画布上同时存在 5+ 个 shape 时，你的方案能不能扛住？请在 README 里说明你的多 shape 方案，以及如果要扩展到 50+ shape 你会怎么改。

3.3 二维语义的几何变换 (~25 min, Must-have)

关键： 3D 模型在画布上要被当成一张 2D 贴图组件。具体来说：

- tldraw 自带的选中 / 平移 / 等比缩放 handle 应该对 shape 生效

- 缩放 shape 时：不是改变 3D 模型在场景里的尺寸，而是改变 shape 在画布上的占地（视觉上像把一张图片拉大），R3F 内部正交相机或视野相应调整以填满新尺寸
- 旋转 shape (tldraw rotate handle)：旋转的是 shape 在 2D 画布上的朝向，不影响 3D 模型自身姿态

3.4 模型姿态调整 (~15 min, Must-have)

在 shape 选中状态下，提供一个独立的旋转控件（建议位于 shape 中央或顶部），点击后绕 Y 轴旋转模型一定角度（比如每次 +45°）：

- 这个旋转改的是 shape 的 `yRotation` prop（不是 shape 在画布上的 2D rotation）
- 视觉上模型像在原地“转一下”，而 shape 的 2D 包围盒不变
- 这个操作必须能被 tldraw 的 `undo/redo` 捕获（Nice-to-have，但请验证一下并在报告里说明结果）

隐性考点： 改 shape props 走 tldraw 的 store 还是绕开它？如果绕开，undo/redo 还能不能正常工作？

3.5 Undo / Redo 验证 (~10 min, Nice-to-have)

- 验证 tldraw 自带的 undo/redo 能正确回放：图片上传 → 转 3D → 移动 → 缩放 → 姿态旋转 → 删除
- 如果某些操作不被捕获（典型嫌疑：异步 API 完成后的 shape 替换），分析原因并写入报告——分析本身也是评分项

Part 4: 总结报告 (建议 20-30 min)

写一份简要总结，写在 README 中：

1. **方案选择说明：** Part 1 调研的方案对比，最终为什么选了现在的方案
2. **架构 & 数据流：** 一张图（手画拍照 / Mermaid 都行）说明 tldraw store ↔ shape ↔ R3F scene ↔ Meshy API 的数据流
3. **Must-have / Nice-to-have 完成情况：** 诚实列出哪些跑通、哪些没做、哪些有思路但来不及实现
4. **遇到的坑：** 开发过程中卡在哪里，怎么排查的，AI 在排查中起了什么作用
5. **如果再给你一天：** 你会优先解决什么问题？（性能？多 Context 共享？Image-to-3D 的实时预览？）描述思路即可，不需要写代码
6. **时间与工具记录：** 总用时、每部分花了多久、用了哪些 AI 工具

交付物清单

以下两项均为硬性要求，缺一不可：

序号	交付物	格式
1	GitHub 仓库（必须）	public 公开仓库；README 写清启动方式
2	Vercel 部署 Demo（必须）	公开可访问的 Vercel 链接，最好绑定到上面的 GitHub 仓库走自动部署
3	调研笔记（Part 1）	写在 README 或单独 Markdown
4	总结报告（Part 4，含时间与 AI 工具记录）	写在 README

Vercel 部署是评分前提：我们要在你提交后直接打开链接体验，本地复现 / 录屏不替代部署。如果 Meshy API Key 涉及秘密，部署时请用 Vercel 环境变量注入，不要把 Key 提交到仓库。

提交方式：把 GitHub 仓库链接 + Vercel Demo 链接发给招聘 HR（HR 会转交给面试官）。

评分维度

维度	权重	说明
探索能力	30%	在没接触过 tldraw 自定义 shape 和 R3F 的前提下，能否独立 ramp-up 并产出可工作 Demo；调研笔记是否体现真实判断
架构合理性	25%	tldraw 与 R3F 的桥接方案、shape props 设计、异步任务与 store 一致性、状态流走向是否合理；多 shape 扩展性思考
Must-have 完成度	20%	优先级清单中红色项是否全部跑通
AI 工作流	15%	全流程 AI 使用是否成熟，能否清晰区分"AI 帮我搞定的事" vs "我自己拍板的判断"
工程质量 & 沟通	10%	TypeScript 类型、组件拆分、README 表达清晰度；时间记录是否真实

注意事项

- 这是一道探索性题目，不是把所有功能塞满才算赢。我们看重的是「在陌生方向上你怎么走出第一步」

- 严格按照「优先级清单」分配时间。**Must-have 全跑通 + 诚实的取舍说明**，远比“什么都做了一半”得分高
- 全程鼓励使用 AI Agent，**诚实记录使用情况——隐瞒反而扣分，把 AI 作为新栈的杠杆是加分项**
- Meshy API 接入卡壳就用本地预设 GLB 兜底，不要在这一步耗时间
- 记得先注册 Meshy 账号并把邮箱发给 HR，这样我们能先帮你充值会员，避免你写到一半发现免费额度不够
- 遇到任何问题（tldraw API 看不懂、R3F 嵌入冲突、坐标系搞不清等），把问题、尝试过的方案、最终结论都记录到报告里——分析过程本身就是评分项